

Traducción de miniC a código ensamblador MIPS

MIPS Práctica de Compiladores



Convocatoria: Mayo 2026

Índice

1. Introducción	3
2. Ficheros entregados	3
3. Análisis léxico	3
3.1. Palabras reservadas	4
3.2. Identificadores	4
3.3. Números enteros	4
3.4. Cadenas de caracteres	5
3.5. Comentarios	5
3.6. Recuperación de errores léxicos	6
4. Análisis sintáctico	6
4.1. Gramática principal	6
4.2. Precedencia y asociatividad	7
4.3. Recuperación de errores sintácticos	7
5. Análisis semántico	7
5.1. Tabla de símbolos	8
5.2. Declaración de variables y constantes	8
5.3. Uso de identificadores	8
5.4. Asignación a constantes	9
5.5. Comprobación del identificador del programa	9
6. Generación de código MIPS	9
6.1. Estructura del código generado	9
6.2. Registros temporales	10
6.3. Asignaciones	10
6.4. Expresiones aritméticas	10
6.5. Operadores relacionales	11
6.6. Sentencia if	11
6.7. Sentencia if-else	11
6.8. Sentencia while	12
6.9. Sentencia print	12
6.10. Sentencia read	12
7. Mejoras implementadas	13
7.1. Sentencia do-while	13
7.2. Operadores relacionales	13
8. Ficheros de prueba	13
9. Ejemplos de funcionamiento	14
9.1. Ejemplo básico	14
9.2. Ejemplo de aritmética	14
9.3. Ejemplo de lectura	15
9.4. Ejemplo con mejoras opcionales	15

9.5. Ejemplo de error semántico	16
10. Conclusiones	16

1. Introducción

El objetivo de esta práctica es implementar un compilador para un lenguaje llamado miniC, que es una versión reducida del lenguaje C. El compilador desarrollado traduce programas escritos en miniC a código ensamblador MIPS.

El proceso de compilación se ha dividido en las fases habituales:

- Análisis léxico, implementado con Flex.
- Análisis sintáctico, implementado con Bison.
- Análisis semántico, mediante una tabla de símbolos.
- Generación de código ensamblador.

El compilador recibe como entrada un fichero fuente con extensión `.mc` y genera por la salida estándar el código ensamblador MIPS.

2. Ficheros entregados

Los ficheros principales entregados son los siguientes:

- `miniC.l`: Especificación Flex del analizador léxico.
- `miniC.y`: Especificación Bison del analizador sintáctico, análisis semántico y generación de código.
- `listaSimbolos.h`: Cabecera de la lista de símbolos proporcionada en prácticas.
- `listaSimbolos.c`: Implementación de la lista de símbolos.
- `listaCodigo.h`: Cabecera de la lista de código proporcionada en prácticas.
- `listaCodigo.c`: Implementación de la lista de código.
- `Makefile`: Fichero para compilar el proyecto.
- `ficheros_de_prueba/`: Carpeta con programas de prueba en miniC.

3. Análisis léxico

El análisis léxico se ha implementado en el fichero `miniC.l` usando Flex. El analizador reconoce:

- Palabras reservadas.
- Identificadores.
- Números enteros.
- Cadenas de caracteres.

- Operadores.
- Operadores relacionales implementados como mejora.
- Separadores y caracteres especiales.
- Comentarios de línea y comentarios multilínea.

3.1. Palabras reservadas

Las palabras reservadas reconocidas son:

Lexema	Token
void	VOID
var	VAR
const	CONST
int	INT
if	IF
else	ELSE
while	WHILE
do	DO
print	PRINT
read	READ

La palabra reservada `do` se ha añadido para implementar la mejora opcional de la sentencia `do-while`.

3.2. Identificadores

Los identificadores se reconocen mediante una expresión regular que permite letras, dígitos y guiones bajos, pero impide que comiencen por un dígito. Además, se controla que no excedan los 32 caracteres con esta función:

```

1 #define LEN_MAX 32
2
3 int comprueba_identificador() {
4     yyval = copia_lexema(yytext);
5     if (yylen > LEN_MAX) {
6         error_lexico("identificador con mas de 32 caracteres");
7     }
8     return ID;
9 }

```

3.3. Números enteros

Los números enteros se reconocen como secuencias de dígitos. Además, se comprueba que sean enteros cuyo valor absoluto esté entre 0 y 2^{31} .

Si se detecta un entero fuera de rango, se emite un error léxico. Esto se hace con esta función:

```

1 #define NUM_MAX "2147483648"
2
3 int comprueba_entero() {
4     char *inicio = yytext;
5     int len;
6     int max_len = strlen(NUM_MAX);
7
8     while (*inicio == '0' && *(inicio + 1) != '\0') {
9         inicio++;
10    }
11
12    len = strlen(inicio);
13    yylval = copia_lexema(yytext);
14
15    if (!(len < max_len || (len == max_len && strcmp(inicio, NUM_MAX)
16    <= 0))) {
17        error_lexico("entero fuera de rango");
18    }
19    return NUM;
20 }

```

3.4. Cadenas de caracteres

La expresión regular de las cadenas de caracteres permite caracteres normales y también secuencias escapadas como `\n`, `\t`, `\r` y `\"`. Para ello se usa la parte `\\["\tnr]` de la expresión regular.

```
1 cadena \\"([^\\""]|\\\\"["\tnr"])*\\"
```

3.5. Comentarios

Se reconocen dos tipos de comentarios:

- Comentarios de línea, que empiezan por `//`.
- Comentarios multilínea, que empiezan por `/*` y terminan por `*/`.

Ambos tipos de comentarios se ignoran durante el análisis léxico. Esto se hace mediante el uso de la condición de arranque `COMENTARIO`:

```

%x COMENTARIO
/*"      BEGIN(COMENTARIO);
"*/"     BEGIN(INITIAL);
[^*\n]+ ;
"*"      ;
\n       ;
«EOF»    {error_lexico("comentario multilínea sin cerrar"); yyterminate();}

```

3.6. Recuperación de errores léxicos

El analizador léxico implementa una recuperación básica en modo pánico. Cuando se detecta un símbolo no reconocido, un identificador inválido o una cadena sin cerrar, se informa del error y se entra en un estado de pánico.

Durante este estado, se descartan caracteres hasta encontrar un posible comienzo de token válido o un separador adecuado. Esto permite que el análisis pueda continuar después de un error léxico:

```
1 %x PANICO
2
3 <PANICO>[ \t\n\r]+      BEGIN(INITIAL);
4 <PANICO>"/"/"[^\n]*     BEGIN(INITIAL);
5 <PANICO>"/*"            BEGIN(COMENTARIO);
6 <PANICO>[a-zA-Z_0-9\";,+\-*/=(){}<>!] { yyless(0); BEGIN(INITIAL); }
7 <PANICO>.               ;
8 <PANICO><<EOF>>         yyterminate();
```

4. Análisis sintáctico

El análisis sintáctico se ha implementado con Bison en el fichero `miniC.y`.

4.1. Gramática principal

La gramática reconoce:

- Declaraciones de variables.
- Declaraciones de constantes.
- Asignaciones.
- Bloques de sentencias.
- Sentencias `if`.
- Sentencias `if-else`.
- Sentencias `while`.
- Sentencias `do-while`.
- Sentencias `print`.
- Sentencias `read`.
- Expresiones aritméticas.
- Expresiones relacionales.

4.2. Precedencia y asociatividad

Para resolver la ambigüedad de las expresiones, se han declarado precedencias y asociatividades en Bison.

Los operadores aritméticos tienen precedencia habitual:

- `*` y `/` tienen mayor precedencia que `+` y `-`.
- `+` y `-` son asociativos por la izquierda.
- `*` y `/` son asociativos por la izquierda.
- El menos unario tiene mayor precedencia que los operadores binarios.

Para ello se ha implementado esta zona del código junto a su implementación en `expression`:

```
1 %left SUMA RESTA
2 %left MULTI DIV
3 %right MENOS_UNARIO
```

Con esto, los operadores `+` y `-` son asociativos por la izquierda, al igual que `*` y `/`. Además, al aparecer `MULTI` y `DIV` después de `SUMA` y `RESTA`, tienen mayor precedencia.

4.3. Recuperación de errores sintácticos

Se ha implementado una recuperación básica de errores sintácticos mediante el token especial `error` de Bison.

La recuperación se realiza principalmente sincronizando con el punto y coma `;` dentro del cuerpo del programa o de una lista de sentencias. De este modo, ante un error sintáctico, el compilador puede descartar tokens hasta encontrar un punto seguro desde el que continuar el análisis.

Un ejemplo de error recuperable es:

```
1 void main () {
2     var int x =
3     print(x);
4 }
```

En este caso, falta el punto y coma después de la declaración de `x`. El analizador informa del error y continúa el análisis tras encontrar un punto de sincronización.

5. Análisis semántico

El análisis semántico se realiza durante el análisis sintáctico, mediante acciones semánticas incluidas en el fichero `miniC.y`.

Para ello se utiliza una tabla de símbolos basada en la implementación proporcionada en el aula virtual.

5.1. Tabla de símbolos

La tabla de símbolos almacena la información de:

- Variables.
- Constantes.
- Cadenas de caracteres.

Cada variable o constante declarada se inserta en la tabla de símbolos. Para evitar colisiones con instrucciones o etiquetas de MIPS, las variables se imprimen posteriormente en ensamblador precedidas por un guion bajo.

Por ejemplo, una variable `miniC`:

```
1 var int x;
```

se representa en MIPS como:

```
1 _x:  
2     .word 0
```

Las cadenas se almacenan con etiquetas de la forma:

```
1 $str1:  
2     .asciiz "Texto"
```

5.2. Declaración de variables y constantes

Cuando aparece una declaración de variable o constante, se comprueba si el identificador ya existe en la tabla de símbolos.

Ejemplo válido:

```
1 var int x;  
2 const int y = 5;
```

Ejemplo no válido:

```
1 var int x;  
2 var int x;
```

En el segundo caso, el compilador informa de un error semántico por redeclaración.

5.3. Uso de identificadores

Cada vez que se usa un identificador en una expresión, asignación, lectura o impresión, se comprueba que haya sido declarado previamente.

Ejemplo no válido:

```
1 void main () {  
2     x = 5;  
3 }
```

El identificador `x` no ha sido declarado, por lo que se emite un error semántico.

5.4. Asignación a constantes

El compilador controla que no se pueda modificar el valor de una constante después de su declaración.

Ejemplo no válido:

```
1 void main () {  
2     const int x = 5;  
3     x = 3;  
4 }
```

También se controla que una constante no pueda utilizarse como destino de una sentencia read.

5.5. Comprobación del identificador del programa

La gramática permite reconocer un identificador después de la palabra reservada `void`, pero semánticamente se comprueba que el identificador del programa sea `main`.

Ejemplo válido:

```
1 void main () {  
2 }
```

Ejemplo no válido:

```
1 void programa () {  
2 }
```

6. Generación de código MIPS

La generación de código se realiza mediante una lista de código formada por operaciones con cuatro campos, siguiendo la estructura proporcionada en el aula virtual.

Cada regla relevante de la gramática devuelve una lista de código. Estas listas se van concatenando hasta formar el código completo del programa.

6.1. Estructura del código generado

El código MIPS generado tiene dos partes principales:

- Sección de datos.
- Sección de código.

La sección de datos contiene:

- Cadenas usadas en el programa.
- Variables y constantes declaradas.

La sección de código contiene las instrucciones MIPS correspondientes a las sentencias miniC.

El esquema general es:

```

1 .data
2
3 # Cadenas del programa
4 $str1:
5     .asciiz "Texto"
6
7 # Variables y constantes
8 _x:
9     .word 0
10
11 .text
12 .globl main
13 main:
14     # instrucciones
15     ...
16     li $v0, 10
17     syscall

```

6.2. Registros temporales

Para evaluar expresiones se utilizan registros temporales de MIPS, principalmente de la familia \$t.

Cuando se evalúa una expresión, el resultado queda almacenado en un registro temporal. Después, ese registro puede usarse para una asignación, una impresión, una condición o una operación aritmética.

6.3. Asignaciones

Una asignación miniC como:

```

1 x = 5;

```

se traduce de forma general a:

```

1 li $t0, 5
2 sw $t0, _x

```

Si la parte derecha es una expresión compuesta:

```

1 x = a + b;

```

primero se genera código para cargar a y b, luego se genera la suma, y finalmente se almacena el resultado en x.

6.4. Expresiones aritméticas

Las expresiones aritméticas generan instrucciones MIPS como:

Ejemplo:

```

1 x = 5 + 3 * 2;

```

La precedencia hace que primero se genere el producto y después la suma.

Operador miniC	Instrucción MIPS
+	add
-	sub
*	mul
/	div
-x	neg

6.5. Operadores relacionales

Como mejora opcional, se han añadido los operadores relacionales:

```
1 < > <= >= == !=
```

Estos operadores se pueden utilizar en expresiones, especialmente como condiciones en `if`, `while` y `do-while`.

El resultado de una expresión relacional es:

- 1, si la comparación es verdadera.
- 0, si la comparación es falsa.

Ejemplo:

```
1 if (x < y) print("x menor que y\n");
```

6.6. Sentencia if

Una sentencia:

```
1 if (x) print("verdadero\n");
```

se traduce generando:

- Código para evaluar la condición.
- Un salto condicional si la condición es cero.
- Código de la sentencia del cuerpo.
- Una etiqueta de fin.

6.7. Sentencia if-else

Una sentencia:

```
1 if (x) print("verdadero\n");
2 else print("falso\n");
```

se traduce usando dos etiquetas:

- Una etiqueta para la rama `else`.
- Una etiqueta para el final de la sentencia.

6.8. Sentencia while

Una sentencia:

```
1 while (x) {  
2     print(x);  
3     x = x - 1;  
4 }
```

se traduce generando:

- Una etiqueta de inicio del bucle.
- Código para evaluar la condición.
- Un salto al final si la condición es falsa.
- Código del cuerpo.
- Un salto al inicio.
- Una etiqueta de fin.

6.9. Sentencia print

La sentencia `print` puede imprimir expresiones enteras y cadenas.

Ejemplo:

```
1 print("x =", x, "\n");
```

Para imprimir cadenas se utiliza la llamada al sistema con código 4:

```
1 li $v0, 4  
2 la $a0, $str1  
3 syscall
```

Para imprimir enteros se utiliza la llamada al sistema con código 1:

```
1 li $v0, 1  
2 move $a0, $t0  
3 syscall
```

6.10. Sentencia read

La sentencia `read` permite leer enteros por teclado.

Ejemplo:

```
1 read(x);
```

Se traduce mediante la llamada al sistema con código 5:

```
1 li $v0, 5  
2 syscall  
3 sw $v0, _x
```

7. Mejoras implementadas

Se han implementado dos mejoras opcionales.

7.1. Sentencia do-while

Se ha añadido la sentencia:

```
1 do statement while (expression);
```

Esta sentencia ejecuta primero el cuerpo y después evalúa la condición. Si la condición es distinta de cero, vuelve al inicio del bucle.

Ejemplo:

```
1 do {  
2     print(i);  
3     i = i - 1;  
4 } while (i);
```

A diferencia de `while`, el cuerpo se ejecuta al menos una vez. La traducción genera primero la etiqueta de inicio, después el código del cuerpo, luego la evaluación de la condición y finalmente un salto al inicio si la condición es verdadera.

7.2. Operadores relacionales

Se han añadido los operadores:

```
1 < > <= >= == !=
```

Estos operadores se han incluido en el no terminal `expression`. No son asociativos y tienen precedencia inferior a los operadores aritméticos. Por tanto, se permite:

```
1 if (x + 1 < y)
```

pero no se permite encadenar comparaciones como:

```
1 x < y < z
```

8. Ficheros de prueba

Se han incluido varios ficheros de prueba para verificar el funcionamiento del compilador.

Fichero	Objetivo
correcto_basico.mc	Comprueba declaraciones, asignaciones, while, if-else y print.
correcto_aritmetica.mc	Comprueba expresiones aritméticas, precedencia y menos unario.
correcto_read.mc	Comprueba lectura de enteros con read.
correcto_mejoras.mc	Comprueba do-while y operadores relacionales.

Fichero	Objetivo
correcto_relacional.mc	Comprueba todos los operadores relacionales implementados.
error_redeclarada.mc	Comprueba error por declarar dos veces el mismo identificador.
error_no_declarada.mc	Comprueba error por usar una variable no declarada.
error_constante.mc	Comprueba error por intentar modificar una constante.
error_main.mc	Comprueba error si el programa no se llama main.
error_sintactico.mc	Comprueba detección y recuperación de errores sintácticos.
error_relacional_no_asociativo.mc	Comprueba que los operadores relacionales no son asociativos.

9. Ejemplos de funcionamiento

9.1. Ejemplo básico

Programa de entrada:

```

1 void main () {
2     var int c = 5;
3     print("Inicio del programa\n");
4     while (c) {
5         print("c = ", c, "\n");
6         c = c - 1;
7     }
8     print("Final\n");
9 }

```

Comando usado:

```

1 ./minic ficheros_de_prueba/correcto_basico.mc > correcto_basico.s
2 spim -file correcto_basico.s

```

Salida esperada:

```

1 Inicio del programa
2 c = 5
3 c = 4
4 c = 3
5 c = 2
6 c = 1
7 Final

```

9.2. Ejemplo de aritmética

Programa de entrada:

```

1 void main () {
2     var int x = 5 + 3 * 2;
3     var int y = -x + 27;
4     print("Resultado =", y, "\n");
5 }

```

Comando usado:

```

1 ./minic ficheros_de_prueba/correcto_aritmetica.mc >
   correcto_aritmetica.s
2 spim -file correcto_aritmetica.s

```

Salida esperada:

```

1 Resultado = 16

```

9.3. Ejemplo de lectura

Programa de entrada:

```

1 void main () {
2     var int x;
3     print("Introduce x:\n");
4     read(x);
5     print("Doble =", x * 2, "\n");
6 }

```

Comando usado:

```

1 ./minic ficheros_de_prueba/correcto_read.mc > correcto_read.s
2 spim -file correcto_read.s

```

Entrada introducida: 7

Salida esperada:

```

1 Introduce x:
2 Doble = 14

```

9.4. Ejemplo con mejoras opcionales

Programa de entrada:

```

1 void main () {
2     var int i = 0;
3     var int suma = 0;
4     do {
5         i = i + 1;
6         suma = suma + i;
7     } while (i < 5);
8     if (i == 5)
9         print("i = 5\n");
10    print("suma =", suma, "\n");
11 }

```


Comando usado:

```
1 ./minic ficheros_de_prueba/correcto_mejoras.mc > correcto_mejoras.s
2 spim -file correcto_mejoras.s
```

Salida esperada:

```
1 i = 5
2 suma = 15
```

9.5. Ejemplo de error semántico

Programa de entrada:

```
1 void main () {
2     var int x;
3     var int x;
4 }
```

Salida esperada:

```
1 ERROR SEMANTICO: identificador ya declarado
2 Analisis terminado con errores
```

En este caso no se genera código MIPS válido, ya que el programa fuente contiene errores.

10. Conclusiones

En este proyecto hemos aprendido de forma práctica cómo funciona internamente un compilador, desde la lectura del programa fuente hasta la generación de código ensamblador. Para ello, hemos dividido el proceso en varias fases: análisis léxico, análisis sintáctico, análisis semántico y generación de código. Esto nos ha permitido entender mejor cómo un lenguaje de alto nivel puede ir transformándose progresivamente en instrucciones más cercanas a la máquina.

También hemos trabajado con herramientas habituales en la construcción de compiladores, como Flex y Bison, viendo cómo se pueden definir tokens, gramáticas, precedencias y acciones semánticas. Además, mediante la tabla de símbolos hemos comprobado la importancia de controlar correctamente las declaraciones, el uso de variables y constantes, y los posibles errores semánticos antes de generar código.

Finalmente, la generación de código MIPS nos ha servido para relacionar las instrucciones del lenguaje miniC con operaciones reales de bajo nivel, como cargas, almacenamientos, saltos, operaciones aritméticas y llamadas al sistema. En conjunto, la práctica nos ha ayudado a comprender mejor el proceso completo de traducción de un programa y la conexión entre los lenguajes de programación y la arquitectura del computador.